

Drift Car Rental

Database design & implementation report

Database module project | 1 October 2026

Implementation snapshot: GitHub branch [codex/mysql-persistence](#), baseline application commit **d683da8**, with the rubric-alignment additions described in the appendix. This report describes the implemented local system, rather than a proposed design.

Relational tables	Foreign keys	CHECK constraints	Fleet vehicles
21	24	32	40

Project purpose

Drift connects a car-rental website to a persistent MySQL database. Customers browse the catalogue, register, sign in, select rental dates and extras, and create reservations. Administrators manage fleet records, review cancellation requests and inspect the live database structure.

Current outcome

The local application stores accounts, vehicles, bookings and payment records in MySQL. Checkout approves payments automatically and saves the reservation; it does not collect card information or charge money. These approvals are excluded from collected-revenue reports.

Report guide

Page	Content
2-3	Architecture, requirements and relational model
4-5	Relationships, integrity, transactions and security
6-7	SQL examples and verification evidence
8	Handover, limitations and project references
9-14	Rubric mapping, normalization and complete ER model

Prepared from the project source, schema and local verification results. Group names and student numbers have not been supplied. The rubric mapping and remaining submission steps are in the appendix.

Architecture & requirements

Layer	Implemented responsibility
Browser / React	Catalogue, registration, account pages, checkout and administration. The browser retains only the checkout draft; the server controls identity and pricing.
Vinext / Node.js	Route handlers validate requests, enforce access rules and execute application workflows. The local build uses Vite and Nitro.
mysql2 / repository	Parameterized SQL, connection pooling and transaction handling.
MySQL / InnoDB	Persistent records, relationships, uniqueness, checks, generated columns and transaction locks.

User roles

Customer: browse vehicles and extras, create reservations, view their own bookings and request cancellation. New registrations receive the customer role. **Administrator:** manage vehicles, add branches/extras, access all bookings, make supported status decisions and inspect the database schema.

Implemented requirements

Requirement	Implementation
Persistence	Stored data remains available across page refreshes and server restarts.
Availability	Server checks date overlap while holding a vehicle row lock.
Pricing	Vehicle and extra prices are read from MySQL and recalculated before booking.
Access control	Server checks sessions, administrator role and booking ownership.
Audit trail	BookingStatusHistory records status changes; CancellationRequest records review decisions.
Database presentation	Database Studio exposes actual metadata, relationships, row counts and SQL definitions to administrators.

Deployment boundary

The application runs at `http://127.0.0.1:3000` on the project computer, connected to local MySQL. GitHub holds the source and schema; it is not a running database or website host. Online deployment is outside the current scope.

Relational model

The schema separates locations, reusable catalogue data, authentication and reservation records. All 21 tables use InnoDB. The inspected local schema contains 177 columns and 43 primary/unique indexes, in addition to its foreign keys and CHECK constraints.

Area	Tables and responsibilities
Locations (3)	Province: province names. City: city within a province. Branch: rental location and contact details.
Fleet (6)	VehicleCategory: body type. VehicleModel: brand/model classification. Vehicle: registered asset, branch, rate and status. VehicleImage: ordered gallery and credits. Feature: reusable feature names. VehicleFeature: vehicle-feature association.
Accounts (4)	Customer: renter profile. AppUser: login and role. UserSession: hashed session tokens. PasswordResetToken: schema support for a future reset workflow.
Reservations (3)	Booking: dates, customer/vehicle snapshots and totals. RentalExtra: available add-ons and pricing. BookingExtra: selected add-ons with historical prices.
Payment and audit (5)	Payment: payment attempts and statuses. CancellationRequest: requests and reviews. BookingStatusHistory: status audit. BookingEmail: email-delivery tracking structure. VehicleReview: one review per booking.

Normalization decisions

Atomic values: features and galleries use child/association tables rather than comma-separated columns.

Many-to-many relationships: VehicleFeature separates reusable feature definitions from individual vehicles; BookingExtra associates reservations with reusable extras.

Reduced repetition: city/province data is separated from branches, and model/category data is separated from individual registered vehicles. This reduces update anomalies. Pages 10-11 show the normalization process and its physical implementation exceptions.

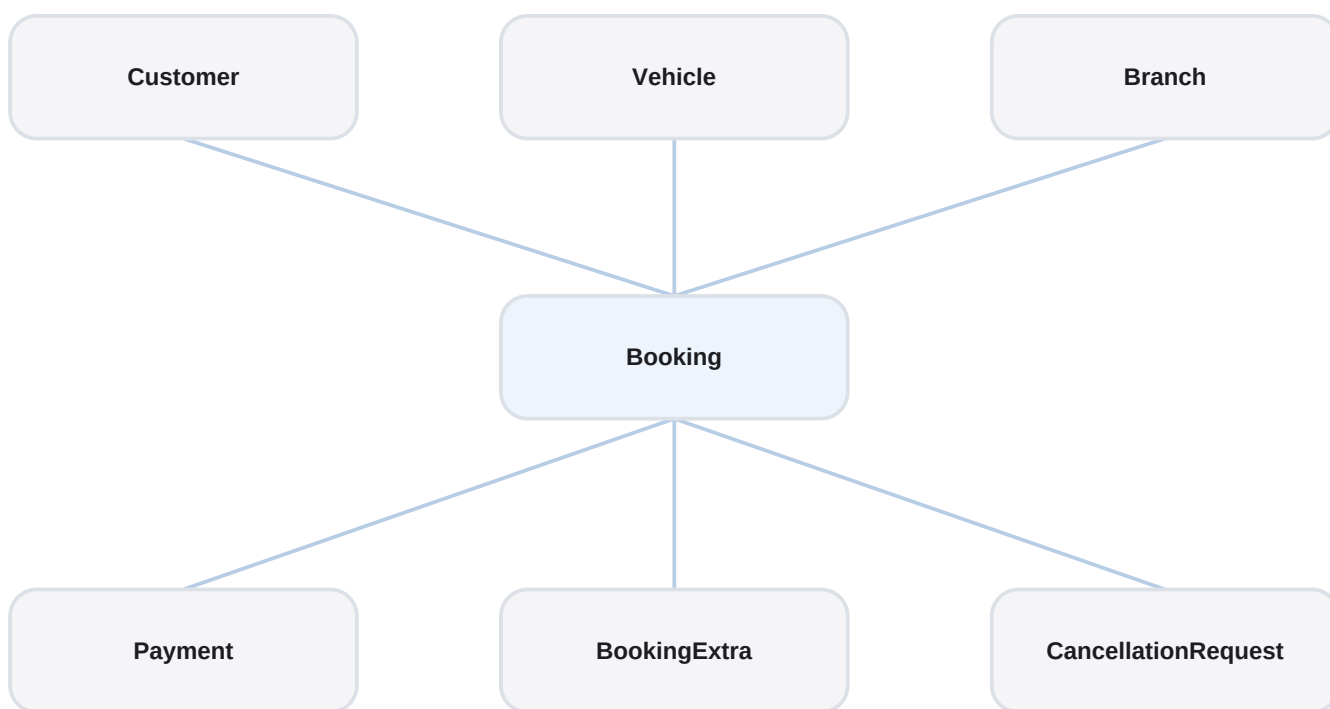
Deliberate historical snapshots: Booking stores the customer name/email, vehicle name and applied daily rate. BookingExtra stores the selected name and price. These values preserve the original reservation when the live catalogue changes; this is intentional duplication for historical accuracy.

Schema does not equal completed functionality

Password-reset and email-tracking tables exist, but the corresponding delivery workflows are not implemented. Review rows can feed catalogue ratings; a complete customer review submission and moderation workflow has not been verified.

Relationships & cardinalities

The diagram highlights the reservation core. It is a conceptual subset, not the full 24-foreign-key physical model. Database Studio provides the complete interactive relationship map and SQL definitions.



Parent records above; reservation and dependent records below.

Relationship	Cardinality and interpretation
Customer - Booking	One customer can have many bookings. Booking.customer_id is nullable in the schema; the application requires a customer-linked account to book.
Vehicle - Booking	One vehicle can appear in many bookings over time. Each booking references one vehicle; overlapping active bookings are rejected.
Branch - Booking	Each booking references one pickup branch and one return branch through separate foreign keys. Each branch can serve many bookings.
Booking - Payment / extras	One booking can have many payment attempts and selected extras. Each BookingExtra references one RentalExtra.
Vehicle - Feature	Many-to-many through VehicleFeature. The composite primary key prevents duplicate associations.
Customer - AppUser	Optional one-to-one link enforced by a unique customer_id. Admin accounts may have no customer link.

Integrity, transactions & security

Mechanism	Concrete example
Primary / foreign keys	VehicleFeature has a composite primary key. Booking references Vehicle, Customer and two Branch roles.
Unique constraints	Registration number, booking reference and checkout idempotency key are unique. A generated nullable key allows only one primary image per vehicle.
CHECK constraints	Vehicle daily_rate must be positive; booking end_date must not precede start_date; ratings must lie between 1 and 5.
Generated columns	Booking rental_days, rental_subtotal and total_cost are computed from dates, the applied rate and extras_total.
Indexes	ix_booking_calendar supports vehicle/status/date lookups; session and reset expiry columns have indexes.

Booking as one transaction

1. Authenticate the customer and validate the request.
2. Lock the selected Vehicle row with `SELECT ... FOR UPDATE`.
3. Check checkout retries, valid branches and overlapping active bookings.
4. Read current rates and extras; reject a changed expected total.
5. Insert Booking, BookingExtra, Payment and BookingStatusHistory.
6. Commit together, or roll back if any step fails.

Availability treats both dates as inclusive. Charged days are $\max(1, \text{end date} - \text{start date})$, so a same-day rental is charged for one day. A cancellation request does not release the vehicle until an administrator approves cancellation.

Why row checks are not enough

A CHECK constraint validates a row; it cannot by itself prevent two concurrent reservations for the same vehicle. The application combines a vehicle lock and overlap query. All future booking writers must follow the same locking convention. Cancellation decisions also update the booking, request and history together.

Implemented security controls

Passwords use salted script hashes. Random session tokens are stored only as SHA-256 hashes, expire after seven days and can be revoked. Cookies are HttpOnly and SameSite=Lax, with Secure enabled under HTTPS. Mutations check the request origin. SQL parameters, server role checks and ownership checks protect the database path. Local credentials remain in an ignored environment file.

SQL examples for presentation

These read-only examples match the supplied schema. Run them in MySQL Workbench after selecting `drift_car_rental`. They are illustrative queries; no result totals below are fabricated. Use the exact identifier case from the imported schema on case-sensitive servers.

1 / Join vehicles to models, categories and branches

```
SELECT v.registration_no, m.brand, m.model_name,
       c.category_name, b.branch_name, v.daily_rate
FROM Vehicle v
JOIN VehicleModel m ON m.model_id = v.model_id
JOIN VehicleCategory c ON c.category_id = m.category_id
JOIN Branch b ON b.branch_id = v.branch_id
WHERE v.is_active = 1
ORDER BY v.daily_rate;
```

2 / Aggregate reservations by status

```
SELECT status, COUNT(*) AS reservation_count,
       COALESCE(SUM(total_cost), 0) AS booking_value
FROM Booking
GROUP BY status
ORDER BY status;
```

Booking value is the sum of reservation prices. It must not be presented as money collected.

3 / List vehicles without conflicting reservations

```
SET @pickup = '2027-01-10';
SET @return = '2027-01-12';
SELECT v.vehicle_id, v.registration_no
FROM Vehicle v
WHERE v.is_active = 1
      AND v.status NOT IN ('Maintenance', 'Rented')
      AND NOT EXISTS (
        SELECT 1 FROM Booking b
        WHERE b.vehicle_id = v.vehicle_id
              AND b.status NOT IN ('Cancelled', 'Completed')
              AND b.start_date <= @return
              AND b.end_date >= @pickup
      );
```

This SELECT illustrates overlap logic. It is not a safe standalone reservation writer: the actual booking workflow performs the check within a transaction while holding the vehicle lock.

4 / Inspect integrity rules

```
SHOW CREATE TABLE Booking;
SHOW INDEX FROM Vehicle;
```

Verification & interface evidence

Verification completed on the local implementation represented by commit 5e51535. The integration script creates temporary records and removes them afterward. These checks establish the tested workflows, not a claim of complete production readiness.

Check	Observed result
TypeScript / local build	npx tsc --noEmit and npm run build:local passed.
Identity and access	Registration, login, session handling, logout and administrator restrictions passed. Schema access returned 401 for guests, 403 for customers and 200 for admins.
Booking integrity	Server pricing, concurrent retry deduplication, overlap rejection, payment-record creation and cancellation flow passed.
Schema and catalogue	21 tables, 24 foreign keys, 32 CHECK constraints and 40 vehicles were confirmed by the local metadata check.
Browser wording	Checkout displayed automatic approval and the no-charge notice; the inspected page contained no demo/simulation wording. No browser errors were reported in that check.
Known test gap	Repository-wide lint is not a clean gate. Load testing, a security audit and a full review of every edge case have not been completed.

The screenshot displays the 'Database studio' interface for a MySQL 8.4.11 database. The sidebar on the left contains navigation links: Dashboard, Vehicles, Bookings, Customers, Reports, and Database (highlighted). The main content area shows a 'LIVE SCHEMA' overview with the title 'Your data. Beautifully connected.' and a description: 'One relational system. Every vehicle, customer and reservation connected through enforced keys.' Below this, four summary cards provide key statistics: 21 Relational Tables (177 defined columns), 445 Stored Records (Exact counts - all tables), 24 Foreign Keys (Database-enforced relationships), and 32 Check Constraints (43 primary / unique indexes). The 'SCHEMA EXPLORER' on the left lists tables under 'FLEET & LOCATIONS': branch (1), city (1), feature (30), province (1), vehicle (40), vehiclecategory (5), and vehiclefeature (154). The 'TABLE SPOTLIGHT' for the 'booking' table shows a 'RELATIONSHIP MAP' with references to bookingemail, bookingextra, and bookingstatushistory.

Figure 1. Database Studio after the visual refinement. This earlier interface capture includes temporary verification records in its row count; it is not a current business-data total.

Handover & remaining work

Run the local system

Install Node.js 22.13 or newer and MySQL 8.4. Import `database/Database_Updated.sql` into an empty schema, install dependencies with `npm ci`, and configure the ignored `.env.local` file using `.env.example`. Do not reimport the fresh-install schema over an existing database.

```
npm run fleet:import
npm run build:local
npm start
```

The fleet importer preserves existing registrations and extras. On a fresh installation, register an account and use `npm run admin:promote -- YOUR-REGISTERED-EMAIL` to grant administrator access. Do not include passwords or session tokens in an assignment submission.

Remaining work and scope

Priority	Action
Before submission	Add the group identification details and review the rubric mapping and normalization exceptions with the lecturer. Rehearse registration, reservation, cancellation, table inspection and SQL queries. Review catalogue rates and branch details.
Known limitations	Email delivery, password reset, email verification and licence verification are not enabled. Some branch, extra and gallery maintenance still requires Workbench.
Optional future work	A real payment provider, online hosting, broader automated testing, lint cleanup and operational backup/restore procedures. No money collection is required for the current automatic-approval scope.

Project sources

All technical findings come from the project artifacts below. The report cites implementation evidence rather than external product claims.

Schema and constraints: [database/Database_Updated.sql](#)

Booking and cancellation transactions: [lib/repository.ts](#)

Authentication and sessions: [lib/server-auth.ts](#)

Architecture and route access: [docs/ARCHITECTURE.md](#)

API contract: [docs/API.md](#)

Integration checks: [scripts/verify-local-mysql.mjs](#)

Current roadmap: [docs/DATABASE_ROADMAP.md](#)

Conclusion: Drift demonstrates a working relational database application with persistent records, explicit relationships, server-enforced permissions and transactional reservation handling. Its strongest database-module evidence is the connection between schema constraints, real application operations and repeatable verification.

Rubric alignment

Source: Database Systems project 2026.pdf, supplied by the group. Assessment weights: project report 40 marks, implementation 50 marks, extra effort 10 marks. The brief does not specify subcriteria weights; this mapping is evidence, not a predicted grade.

Requirement	Implementation / report evidence
Create/add a table	Admin > Database > Manage your tables creates actual MySQL tables. Items 1 and 4 in the brief repeat this requirement.
Add/update data	Vehicle forms and custom-table forms save records to MySQL.
Delete records	Custom-table rows can be physically deleted. Vehicle removal archives assets to preserve rental history.
Remove a table	An administrator types the custom table name to confirm DROP TABLE. Core tables are protected.
Minimum four reports	Booking value, fleet status, booking status and top vehicles, each downloadable as PDF. The full summary is additional.
System overview	Pages 1-2 describe the service setting, roles and architecture.
ER model and tables	Page 3 lists all core tables. Pages 12-14 represent all 24 foreign-key relationships across the 21 core entities.
UNF to 3NF	Pages 10-11 show the decomposition process, dependencies and implementation exceptions.

Presentation readiness

Table creation/removal needs the dedicated app account to have CREATE and DROP privileges. Run scripts/enable-table-management.ps1 locally, then scripts/verify-table-management.mjs. On the development computer, the permissions were enabled and the automated create/insert/update/delete/drop cycle passed on 1 October 2026, including admin-only access and core-table protection. Other installations still require the permission step.

Group and submission requirements

The brief calls for five members, including at least one student from each of ICT, Data Science and Computer Science. Member names, student numbers and streams have not been supplied. Add these before submission. Submit the report and system through Moodle on 12 October 2026; GitHub is not Moodle submission. Presentations are listed for 11-12 October, 09:00-17:00; confirm the group slot with the lecturer.

Normalization: UNF to 2NF

This worked design example starts from a hypothetical rental worksheet, not actual customer records. A booking may contain multiple extras, and a vehicle may have several features and photographs. Braces below represent repeating groups, not SQL columns.

Unnormalized form (UNF)

```
RENTAL_SHEET(BookingID, CustomerID, CustomerName, Email,
  VehicleID, Registration, ModelID, ModelName, CategoryID,
  CategoryName, BranchID, CityID, CityName, ProvinceID,
  ProvinceName, StartDate, EndDate, AppliedRate,
  {ExtraID, ExtraName, UnitPrice, Quantity},
  {FeatureID, FeatureName}, {ImageID, ImageURL})
```

Repeating extras, features and images prevent a single atomic value in each cell. Repeating customer, vehicle and location descriptions also produces update, insertion and deletion anomalies: changing one city name would require editing many rental rows.

First normal form (1NF): remove repeating groups

Create one BookingFlat row per BookingID and separate atomic relations: BookingExtraFlat(BookingID, ExtraID, ExtraName, UnitPrice, Quantity); VehicleFeatureFlat(VehicleID, FeatureID, FeatureName); and VehicleImage(ImageID, VehicleID, ImageURL). Candidate keys for the two association relations are their paired identifiers. Values are atomic, but partial and transitive dependencies remain.

Functional dependencies used

```
BookingID -> CustomerID, VehicleID, dates, AppliedRate
CustomerID -> CustomerName, Email
VehicleID -> Registration, ModelID, BranchID
ModelID -> ModelName, CategoryID
CategoryID -> CategoryName
BranchID -> CityID; CityID -> CityName, ProvinceID
ProvinceID -> ProvinceName
ExtraID -> current ExtraName, current Price
(BookingID, ExtraID) -> historical UnitPrice, Quantity
FeatureID -> FeatureName
```

Second normal form (2NF): remove partial dependencies

In VehicleFeatureFlat, FeatureName depends only on FeatureID, not on the full (VehicleID, FeatureID) key. Move the name to Feature and retain VehicleFeature(VehicleID, FeatureID). In BookingExtraFlat, move the current extra definition to RentalExtra; retain the booking-specific quantity and historical unit price in BookingExtra. The remaining non-key attributes in these association relations depend on the whole candidate key.

BookingFlat has a single-column key, so it has no partial-key dependency; it can still violate 3NF through transitive dependencies. That is addressed next.

Normalization: 3NF & implementation

Third normal form (3NF): remove transitive dependencies

Split current descriptive facts from BookingFlat: Customer owns profile details; Vehicle owns registration and asset details; VehicleModel owns brand/model/category; VehicleCategory owns its name; Branch owns its city reference; City owns its province reference; Province owns its name. Booking refers to the customer, vehicle and pickup/return branches instead of using their current descriptions as authoritative data.

Normalized relation	Key dependency
Customer	customer_id -> current name, email, phone and licence details
Vehicle / VehicleModel	vehicle_id -> model_id, branch_id, registration, asset attributes; model_id -> brand, model_name, category_id
VehicleCategory	category_id -> category_name
Branch / City / Province	branch_id -> branch attributes, city_id; city_id -> city_name, province_id; province_id -> province_name
Feature / VehicleFeature	feature_id -> feature_name; association key is (vehicle_id, feature_id)
RentalExtra / BookingExtra	extra_id -> current extra definition; (booking_id, extra_id) -> selected quantity and price at booking
Booking	booking_id -> customer_id, vehicle_id, branch references, dates and agreed rental rate

Lossless reconstruction and integrity

Each separated entity retains its primary key, and referencing relations retain foreign keys. Joining an association to a referenced primary/unique key reconstructs its descriptions without multiplying rows unexpectedly. Unique business identifiers, such as registration_no and extra_code, prevent duplicate definitions. The complete implemented schema extends this core with account/session, payment, cancellation, email and review entities.

Physical exceptions must be explained

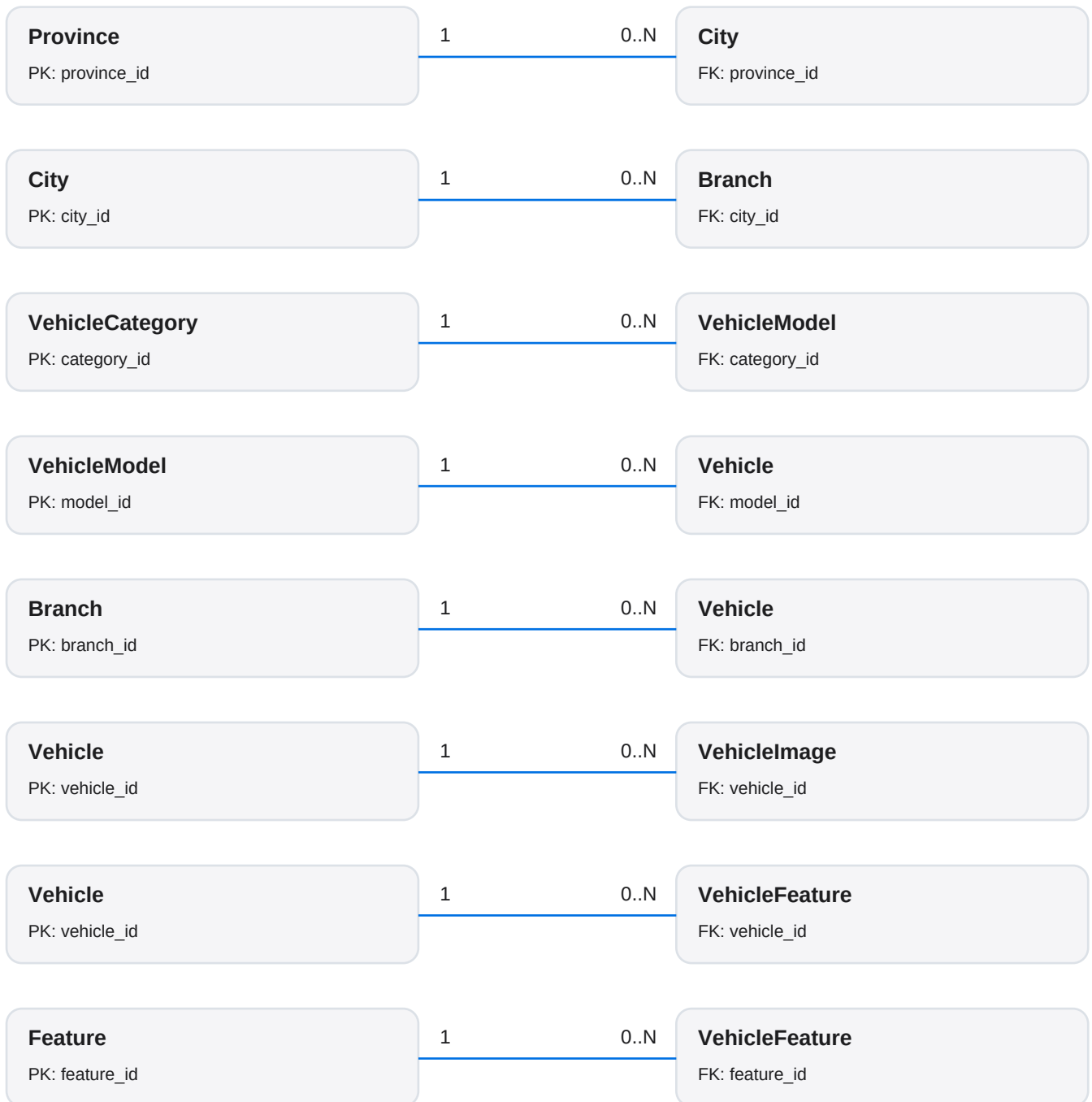
The physical implementation deliberately stores historical name/email snapshots in Booking and an extra-name snapshot in BookingExtra. These record the agreed facts at reservation time, rather than the current profile/catalogue value. Generated subtotals and total_cost, plus the transaction-maintained extras_total, also materialize derived values. They are not an unqualified claim that every stored physical attribute is strict 3NF.

The normalized logical decomposition above satisfies the teaching process; the group should explain the historical/derived implementation exceptions to the lecturer. If the lecturer requires a strictly normalized physical schema with no such materialization, these columns require a separate migration and application refactor. User-created tables must also be designed with appropriate dependencies; the table builder does not automatically prove 3NF.

Complete ER model / 1 of 3

These relationship panels collectively show every foreign key in the 21-table core schema. Repeated boxes denote the same entity. The full field definitions are in database/Database_Updated.sql; the table inventory is on page 3.

Read left to right: parent entity and referenced primary key, then child entity and foreign key. 1 means exactly one; 0..1 means optional one; 0..N means zero or many. Each child row references the parent with the indicated optionality. Custom administrator-created tables are independent additions.

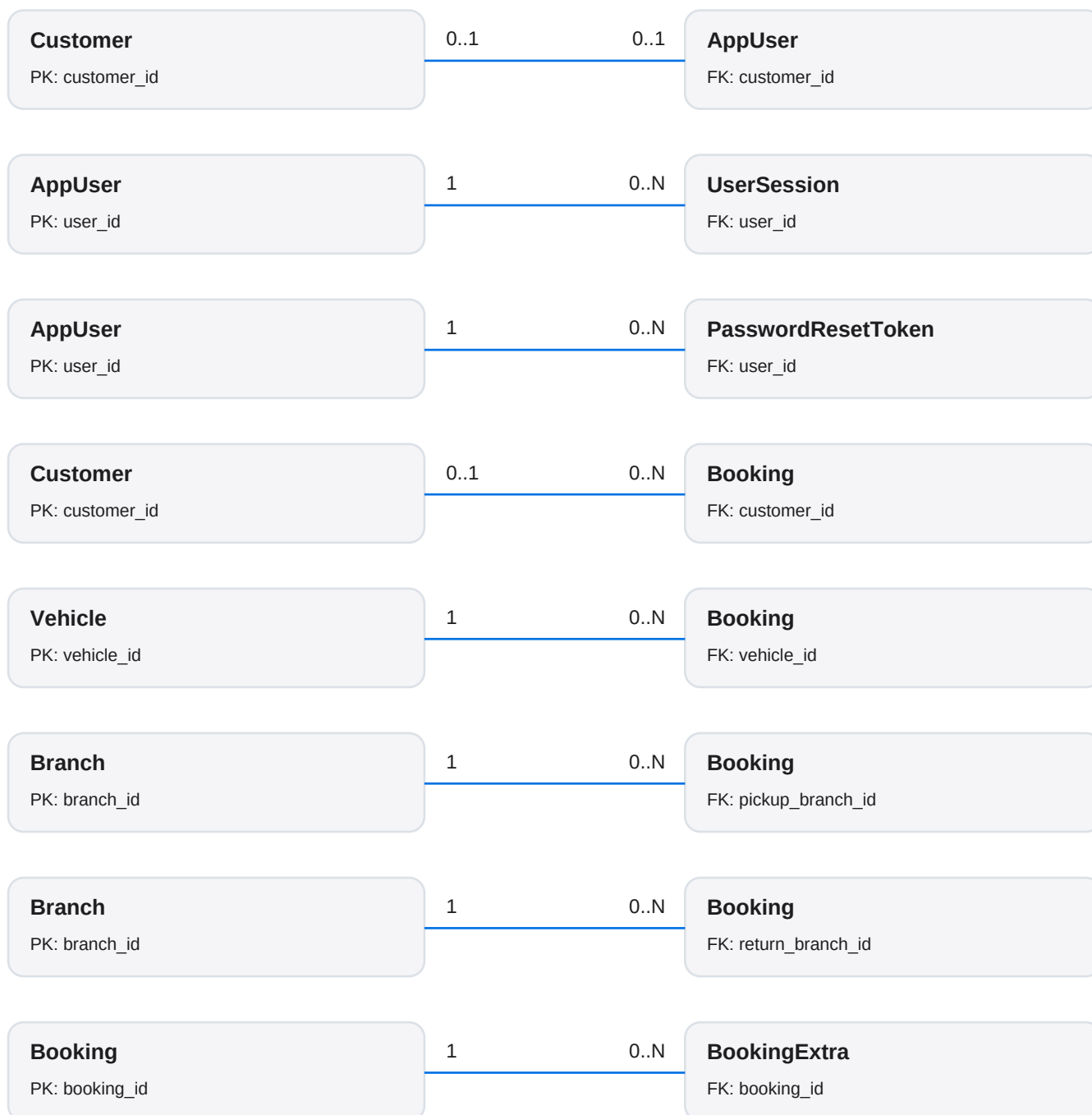


Relationships 1-8 of 24. Generated from the implemented CREATE TABLE definitions.

Complete ER model / 2 of 3

These relationship panels collectively show every foreign key in the 21-table core schema. Repeated boxes denote the same entity. The full field definitions are in database/Database_Updated.sql; the table inventory is on page 3.

Read left to right: parent entity and referenced primary key, then child entity and foreign key. 1 means exactly one; 0..1 means optional one; 0..N means zero or many. Each child row references the parent with the indicated optionality. Custom administrator-created tables are independent additions.

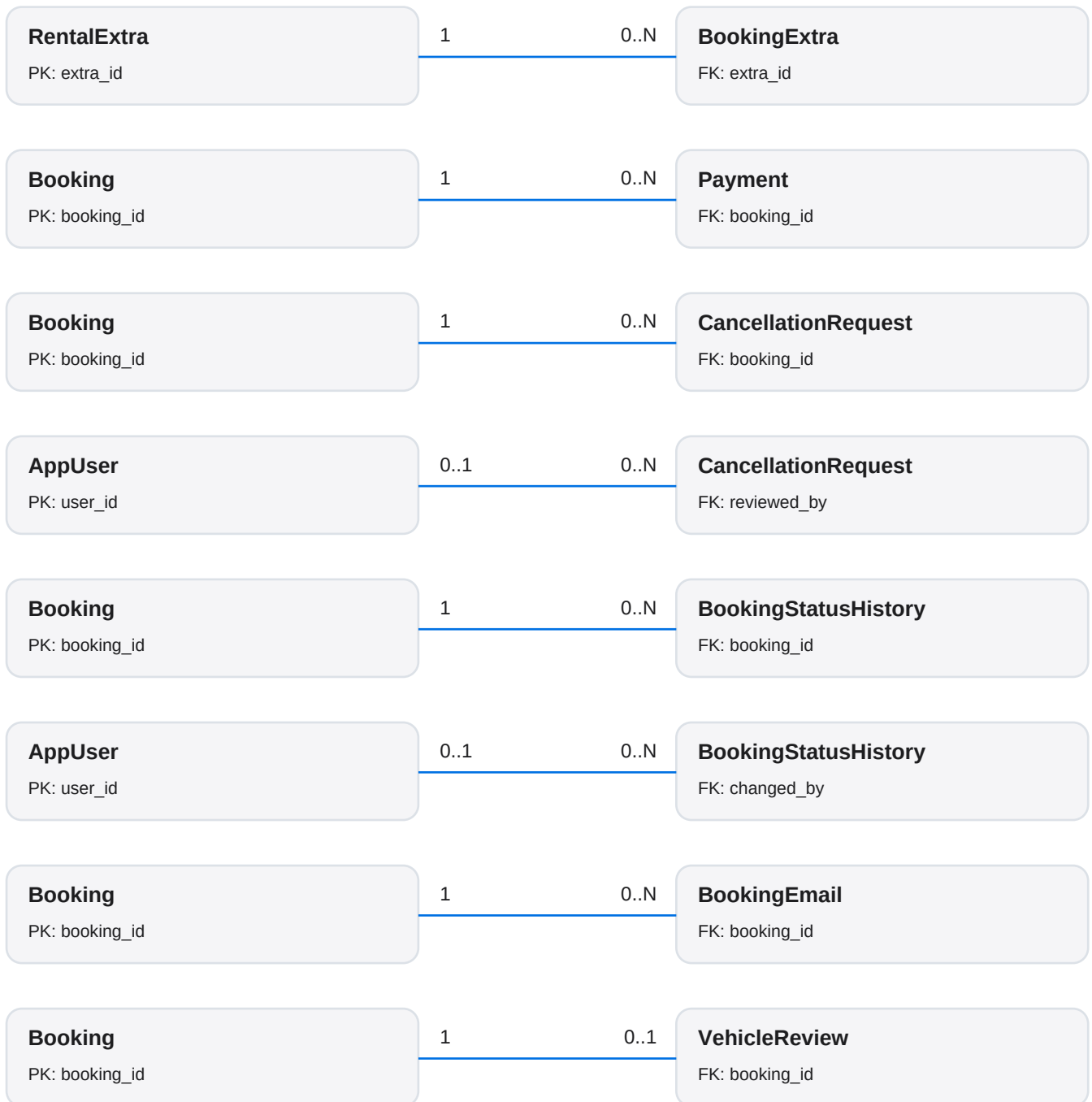


Relationships 9-16 of 24. Generated from the implemented CREATE TABLE definitions.

Complete ER model / 3 of 3

These relationship panels collectively show every foreign key in the 21-table core schema. Repeated boxes denote the same entity. The full field definitions are in database/Database_Updated.sql; the table inventory is on page 3.

Read left to right: parent entity and referenced primary key, then child entity and foreign key. 1 means exactly one; 0..1 means optional one; 0..N means zero or many. Each child row references the parent with the indicated optionality. Custom administrator-created tables are independent additions.



Relationships 17-24 of 24. Generated from the implemented CREATE TABLE definitions.